Node Package Manager

Pacotes Especiais Node.js

Objetivos de Aprendizagem

- Utilizar o gerenciador de pacotes NPM
- Identificar funções básicas do Express

Agenda

- Instalação Node.js
- NPM
- Módulos típicos
- *Framework* Express

Node Package Manager

- Sistema de gerenciamento de pacotes do Node.js
- Similar ao pip do Python, Apt do Ubuntu, dentre outros
- Realiza todo o gerenciamento (publicação, instalação, exclusão, etc) no sistema público
- Normalmente, o NPM é instalado junto com Node.js
- Para verificiar a instalação e versão basta digitar no console

```
node -v
npm -v
```

Node Version Manager

- O Node.js disponibiliza ainda um utilitário para o gereciamento de versões
  - `nvm` (OSX e Linux)
  - `nodist` ou `nvm-windows` (Windows)
- Controladores de versões normalmente são instalados a parte
- Permite instalar e alternar entre diversas versões para testes
- Uso do NVM é feito via linha de comando

npm init

- Configuração inicial dos requisitos de um projeto Node.js novo ou existente
- Dentre outras ações cria um arquivo package.json
- O comando deve ser executado já na pasta do projeto

```
npm init
```

package.json

- Arquivo que contém as informações sobre o gerenciamento de pacotes da aplicação
- Lista todas as dependências de pacotes, inclusive versões, em um único arquivo
- Torna a estrutura da aplicação replicável em servidores de produção ou testes

Exemplo

```
{
  "name": "@slidev/theme-seriph",
  "version": "0.25.0",
  "description": "Seriph theme for Slidev",
  "author": "antfu <anthonyfu117@hotmail.com>",
  "license": "MIT",
  "funding": "https://github.com/sponsors/antfu",
  "homepage": "https://sli.dev",
  "repository": {
    "type": "git",
    "url": "https://github.com/slidevjs/themes"
  },
  "keywords": [
    "slidev-theme",
    "slidev"
  ],
  "engines": {
    "node": ">=14.0.0",
    "slidev": ">=v0.47.0"
  },
  "slidev": {
    "defaults": {
      "fonts": {
        "mono": "PT Mono",
        "sans": "PT Serif",
```

Como criar o package.json?

```
npm init
```

Fornecer as informações iniciais

- Nome
- Versão
- Descrição
- Palavras Chaves
- Autor
- Etc

Como instalar as dependências de um projeto?

- `npm install` instala pacotes e as dependências
- Quando executado sem parâmetros instala **todas** as dependências listadas no arquivo `package.json`

```
npm install
```

Como instalar um pacote específico?

- `npm install <module>` instala o pacote especificado e as dependências
- O arquivo `package.json` é atualizado automaticamente

```
npm install lodash
npm install nodemon
```

* Usando uma única linha de comando

```
npm install lodash nodemon
```

Instalação Global

- `npm install <module> -g` instala o pacote especificado e as depedências globalmente
- O arquivo `package.json` é atualizado automaticamente
- Em ambientes de nuvem é pouco utilizado

Resumo NPM

Comando	Ação
`npm init`	Cria o arquivo `package.json` personalizado
`npm init -y`	Cria o arquivo `package.json` padrão
`npm install <package>`	Insere o pacote especificado em `package.json`
`npm install <package> -g`	Instala glogalmente o pacote especificado
`npm install`	Instala todas as dependências contidas em `package.json`

Nodemon is a tool that helps develop node.js based applications by automatically restarting the node application when file changes in the directory are detected.

Nodemon

- Reinicia a aplicação quando o código é modificado
- Mesmo que a aplicação seja finalizada sem erros o monitoramento continua e reinicia a aplicação
- Substitui o comando `node` para inicializar as aplicações
- Não demanda não modificação no código para sua utilização

```
PM2 is a daemon process manager that will help you manage and
keep your application online 24/7
```

PM2

- Realiza monitoramento similar ao nodemon, porém mais completo
- Permite o monitoramento de diversos microserviços em conjunto
- Instalação `npm install pm2 -g`
- Inicia uma aplicação `pm2 start app.js`
- Monitora todos as aplicações inicializadas com pm2 `pm2 monit`

Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

Express

- *Framework* para desenvolvimento web com Nodejs bastante popular
- Provê um servidor web através do módulo `http`
- Utiliza-se de funções de *middlewares* para a criação das funcionalidades de um site ou aplicação
- Provê ainda a funcionalidade de *templates*

Exemplo de Aplicação (`node-http`)

```
const http = require('http');
const server = http.createServer((request, response) => {
    const method = request.method;
    const url = request.url;
    response.setHeader('Content-type', 'text/html');
    response.write('<!DOCTYPE html><html lang="pt-br">');
    response.write('<head><meta charset="UTF-8"></head>');
    response.write('<body>');
    response.write('<h1>Dados da requisição</h1><br>');
    response.write('<hr>');
    response.write('Método da requisição: ' + method + '<br>'); response.write('URL da requisição:' + url + '<br>');
    response.write('</body></html>');
    response.end()
});
server.listen(3000)
```

Use the application generator tool, express-generator, to quickly create an application skeleton.

Express application generator

Gerador de Aplicações Rápidas

- A estrutura de uma aplicação pode ser criada rapidamente com `express-generator`
- Instalação via NPM

```
npm install express express-generator
npm install
```

Usando o `express-generator`

- Com `express` e o `express-generator` instalados
- Para executar diretamente um pacote, utiliza-se o comando `npx`
- Executa-se o `express-generator` com NPX, conforme mostrado ao lado
- Execute o comando na pasta do projeto a ser criado

```
npx express-generator --view=ejs app1
```

- Cria ```app1``` com *template engine* EJS

```
npm install
npm start
```

- Instala as dependências e inicia a aplicação

Exercício 1

- Utilizar o NPM para criar o arquivo `package.json` da aplicação `node-http`, versão 1.0
- Instalar os pacotes:
  - Nodemon
  - Http
- No arquivo `index.js` existente, inserir o código do slide anterior
- Rode a aplicação (`node index.js`)
- Pare a aplicação (CTRL + D)
- Utilize o nodemon para rodar a aplicação novamente (`nodemon index.js`)
- Modifique o código em `index.js` (linha XXX, por exemplo)

Exercício 2

- Instalar os pacotes:
  - nodemon
  - express
  - express-generator
- Utilize o EG para criar a estrutura básica da aplicação **app1**
- Inicie a aplicação utilizando a linha de comando (node, NPM, nodemo ou pm2)
- Verifique a seção *scripts* do arquivo `package.json`
- Modifique o *script start* para que a aplicação seja iniciada utilizando um monitor como Nodemon ou pm2

Exercício 2

- Utilize o nodemon para rodar a aplicação novamente (`nodemon index.js`)
- Modifique o código em `index.js` (linha XXX) pela seguinte linha:

```
res.render('index', {title: 'Express', mysite: 'My first Express Site'})
```

- Adicione a linha abaixo ao final do arquivo ```index.ejs``` dentro do diretório *views*

```
p #{mysite}
```

- Execute a aplicação - O que acontece?

Referências

- NPM
- Nodemon
- PM2
- Express